1. DSA NOTES — 1. Arrays (Lists)

1. What is an Array (List)?

A list is a collection of elements stored in a single variable, accessed by index. In Python, lists are dynamic arrays (resizable, can hold mixed types).

Indexing always starts from 0. Negative indexing (-1) accesses from the end.

2. Example

```
arr = [10, 20, 30, 40, 50]
#idx:   0   1   2   3   4
print(arr[2])   # 30
print(arr[-1])  # 50 (last)
```

3. Characteristics

- Dynamic size (grows/shrinks automatically)
- Can store mixed data types
- Elements stored contiguously (implementation detail)
- Random access in O(1) time
- Duplicate elements allowed

4. Why Lists?

- Random access O(1)
- Easy to traverse/implement
- Cache friendly
- Base for Stack, Queue, Heap etc.

5. Declaration Examples

```
arr = []                    # empty list
arr = [1, 2, 3, 4, 5]
arr = [0] * 5               # [0,0,0,0,0]
arr = list(range(5))        # [0,1,2,3,4]
arr2 = [x*x for x in range(5)]  # comprehension
```

6. Accessing Elements

Formula: arr[i] gives element at index i (O(1))

```
arr = [10, 20, 30, 40, 50]
arr[2]         # 30
arr[0]         # 10 (first)
arr[-1]        # 50 (last)
arr[1:4]       # [20,30,40] slicing
```

7. Basic Operations

```
# Traversal            O(n)
for x in arr: print(x)

# Insertion (at pos)    O(n)
arr.insert(pos, x)
arr.append(x)           # at end O(1)

# Deletion (at pos)     O(n)
arr.pop(pos)
arr.remove(value)       # first match

# Search (linear)       O(n)
idx = arr.index(key) if key in arr else -1

# Update                O(1)
arr[pos] = new_value
```

8. Types of Arrays

- 1D: arr = [1,2,3,4,5]
- 2D (matrix): arr = [[1,2,3],[4,5,6]]
- 3D: nested lists of lists
- Jagged: sub-lists of different lengths

```
mat = [[1,2,3],[4,5,6]]
mat[1][2]   # 6  (row 1, col 2)
```

9. Common Problems

- Find Max / Min element
- Find 2nd Largest / Smallest
- Check if array is sorted
- Remove duplicates (sorted array)
- Left/Right Rotate array
- Move all zeros to end
- Union / Intersection of two arrays
- Kadane's Algorithm (Max Subarray Sum)

10. Time & Space Complexity

Operation	Time	Space
Access	O(1)	O(1)
Search	O(n)	O(1)
Insert(end)	O(1)*	O(1)
Insert(pos)	O(n)	O(1)
Delete	O(n)	O(1)
Traversal	O(n)	O(1)

11. Advantages / Disadvantages

+ Random access, dynamic size, easy to use, built-in methods

− Insertion/Deletion in middle costly, wastage over-allocation

2. DSA NOTES — 2. Strings

1. What is a String?

A string is a sequence of characters. In Python, strings are **immutable** — any modification creates a new string.

Strings act like read-only lists of characters — use slicing, not index assignment.

2. Representation

```
s = "hello"
#idx: 0 1 2 3 4
len(s)      # 5
s[0]        # 'h'
s[-1]       # 'o'
```

3. Declaration Examples

```
s = "hello"
s = str()           # empty string
s = "".join(["h","i"])  # 'hi'
lst = list(s)       # to char list
```

4. Input / Output

```
s = input("Enter string: ")
print(s)
print(f"Value: {s}")   # f-string
```

5. Common String Methods

```
s.lower(); s.upper()          O(n)
s.strip()                     O(n)
s.find(sub); s.index(sub)     O(n)
s.replace(old, new)           O(n)
s.split(sep); sep.join(lst)   O(n)
s.count(sub)                  O(n)
s.startswith(x); s.endswith(x) O(n)
s.isalpha(); s.isdigit()      O(n)
s + t          # concatenation O(n)
s * 3          # repetition
```

6. Common Operations

Op	Desc	Time
Traversal	Access each char	O(n)
Concatenation	Join two strings	O(n)
Comparison	s1 == s2 / s1 < s2	O(n)
Reversal	s[::-1]	O(n)
Slicing	s[a:b]	O(n)

7. Important Algorithms

```
# Reverse a string
rev = s[::-1]

# Check palindrome
is_pal = s == s[::-1]

# Check anagram
is_ana = sorted(s1) == sorted(s2)

# Count vowels
vowels = sum(1 for c in s if c in "aeiouAEIOU")

# Remove duplicates (keep order)
seen = set()
res = "".join(c for c in s if not (c in seen or seen.add(c)))
```

8. Examples

Task	Input	Result
Length	hello	5
Reverse	hello	olleh
Palindrome	madam	True
Anagram	listen/silent	True
Vowels	abhishek	3

9. Complexity Table

Op	Time	Space
Access	O(1)	O(1)
Traversal	O(n)	O(1)
Search	O(n)	O(1)
Concatenation	O(n)	O(n)
Reversal	O(n)	O(n)
Comparison	O(n)	O(1)

10. Tips

- Strings are immutable — repeated concatenation in a loop is O(n²); use ''.join(list)
- Use f-strings for formatting
- Practice: palindrome, anagram, pattern matching
- Know library methods well — huge time-saver in interviews

3. DSA NOTES — 3. Sliding Window

1. What is Sliding Window?

A technique to solve array/string problems by maintaining a window (subarray/substring) and sliding it to explore the solution efficiently.

Reduces time complexity by avoiding recomputation.

2. Why Use It?

- Reduces time complexity
- Avoids recomputation
- Useful for range/subarray/substring problems
- Optimizes brute force O(n²) → O(n)

3. How Does It Work?

- Step 1: Define left, right pointers
- Step 2: Expand window (right++)
- Step 3: Update result per condition
- Step 4: Shrink window (left++) if invalid
- Step 5: Repeat till right reaches end

4. Generic Template

```
left = 0
result = 0
for right in range(n):
    add(arr[right])           # expand
    while window_invalid():
        remove(arr[left])     # shrink
        left += 1
    result = max(result, right-left+1)
```

5. Fixed Size Window

Max sum subarray of size k:

```
def max_sum_k(arr, k):
    window_sum = sum(arr[:k])
    best = window_sum
    for right in range(k, len(arr)):
        window_sum += arr[right] - arr[right-k]
        best = max(best, window_sum)
    return best

# arr=[2,1,5,1,3,2], k=3 -> 9
```

6. Variable Size Window

Smallest subarray with sum >= target:

```
def min_len(arr, target):
    left = total = 0
    best = float('inf')
    for right in range(len(arr)):
        total += arr[right]
        while total >= target:
            best = min(best, right-left+1)
            total -= arr[left]
            left += 1
    return best if best != float('inf') else 0
```

7. Common Problems

- Max Sum/Average of Subarray Size K
- Longest Substring Without Repeating Characters
- Smallest Subarray with Sum >= K
- Fruits into Baskets (at most 2 distinct)
- Number of Occurrences of Anagrams
- Max Consecutive Ones (after replacement)
- Longest Repeating Character Replacement

8. Comparison

Technique	Best For	Time	Space
Brute Force	small input	O(n²)+	O(1)
Sliding Window	range/subarray	O(n)	O(1)
Prefix Sum	sum queries	O(n)+O(1)q	O(n)

9. Tips

- Think: when to expand (right++), when to shrink (left++)
- Initialize window & result correctly
- Dry run on small examples
- Choose Fixed or Variable window as per problem

Time Complexity: O(n) — each element visited at most twice. Space: O(1) mostly.

4. DSA NOTES — 4. Two Pointers

1. What is Two Pointers?

Technique using two indices to traverse a data structure (arrays/strings) to solve problems efficiently.

Helps solve problems in O(n) or O(n log n) time with O(1) space.

2. Why Use Two Pointers?

- Reduces time complexity
- Eliminates need for extra space
- Efficient for sorted arrays/strings
- Great for pairs/triplets, partitions

3. Types

- Opposite Direction (start & end)
- Same Direction (sliding window)
- Slow & Fast Pointers (cycle detection)

4. How It Works (Opposite Direction)

```
arr = [2,4,6,8,10,12,14]
left, right = 0, len(arr)-1
#      ^left                ^right
```

5. Generic Template

```
left, right = 0, n - 1
while left < right:
    if condition:
        # move pointers accordingly
        left += 1
    else:
        right -= 1
return result
```

6. Common Problems

- Pair with given sum (sorted array)
- Remove duplicates from sorted array
- Palindrome check
- Merge two sorted arrays
- Container With Most Water
- 3Sum / 4Sum (sort + two pointers)
- Trapping Rain Water
- Dutch National Flag (partition array)

7. Examples

Problem	Input	Result
Pair sum = 16	[2,4,6,8,10,12,14]	(2,14)
Remove dup	[1,1,2,2,2,3,3,4]	[1,2,3,4]
Palindrome	racecar	True
Most Water	[1,8,6,2,5,4,3,7]	49

8. Code Example (Pair with Sum)

```
def pair_with_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return (arr[left], arr[right])
        elif s < target:
            left += 1
        else:
            right -= 1
    return None
```

9. Slow & Fast Pointers (Cycle Detection)

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
# Time O(n)  Space O(1)
```

10. When to Use

- Array is sorted
- Need pairs/triplets with given condition
- Need to reduce space complexity
- Working with linked lists (slow-fast)

Two Pointers = Simplicity + Efficiency. Master this to solve interview problems easily!

5. DSA NOTES — 5. Greedy Approach

1. What is Greedy Approach?

An algorithmic paradigm that builds a solution step by step, choosing the locally optimal choice at each step, hoping for a global optimum.

Take the best choice at each step and keep going. Don't look back!

2. Characteristics

- Locally optimal choice each step
- Choice is irrevocable
- Reduces time complexity
- Optimal for greedy-choice + optimal substructure problems

3. When to Use Greedy?

- Problem has optimal substructure
- Problem has greedy-choice property
- Best immediate decision leads to global optimum

Greedy-Choice Property: A globally optimal solution can be reached by selecting the best choice at each step.

4. Generic Template

```
def greedy(candidates):
    candidates.sort(key=...)   # sort if needed
    result = []
    for c in candidates:
        if is_feasible(c, result):
            result.append(c)
    return result
```

5. Common Problems Solved

- Activity Selection
- Fractional Knapsack
- Huffman Coding
- Dijkstra's Algorithm
- Minimum Spanning Tree (Kruskal/Prim)
- Job Sequencing with Deadlines
- Candy Distribution
- Coin Change (Min Coins - canonical)
- Interval Scheduling
- Gas Station

6. Example: Activity Selection

```
def activity_selection(activities):
    # activities: list of (start, finish)
    activities.sort(key=lambda x: x[1])
    result = [activities[0]]
    last_finish = activities[0][1]
    for s, f in activities[1:]:
        if s >= last_finish:
            result.append((s, f))
            last_finish = f
    return result
```

7. Example: Fractional Knapsack

```
def fractional_knapsack(items, W):
    # items: list of (weight, value)
    items.sort(key=lambda x: x[1]/x[0], reverse=True)
    total = 0.0
    for w, v in items:
        if W >= w:
            W -= w; total += v
        else:
            total += v * (W / w)
            break
    return total
```

8. Example: Coin Change (Min Coins)

```
def min_coins(coins, amount):
    coins.sort(reverse=True)
    count = 0
    for c in coins:
        while amount >= c:
            amount -= c; count += 1
    return count
# Works only for canonical coin systems
```

9. Greedy vs Dynamic Programming

Aspect	Greedy	DP
Approach	local best choice	solve all subproblems
Choices	irrevocable	reversible
Optimality	not always optimal	always optimal
Time	usually lower	usually higher

10. Advantages / Pitfalls

+ Simple, fast, works well for real-world problems

− Doesn't always give optimal solution; greedy-choice property hard to prove

6. DSA NOTES — 6. Stack

1. What is Stack?

A linear data structure that follows **LIFO** (Last In First Out) order. Insertion/deletion allowed only at one end called TOP.

> *Like a plate stack in canteen — last plate placed is the first one removed.*

2. Representation

```
stack = [10, 20, 30, 40]
#                         ^ TOP (index -1)
# TOP element    = 40 (stack[-1])
# BOTTOM element = 10 (stack[0])
```

3. Basic Operations

- push(x) → add element x on top
- pop() → remove & return top element
- peek()/top() → return top without removing
- isEmpty() → check if stack is empty
- size() → number of elements

4. Implementation using List

```
stack = []

# Push - O(1)
stack.append(x)

# Pop - O(1)
if stack:
    x = stack.pop()
else:
    print("Stack Underflow")

# Peek - O(1)
top = stack[-1] if stack else None

# isEmpty
is_empty = len(stack) == 0
```

5. Implementation using collections.deque

```
from collections import deque
stack = deque()
stack.append(10)       # push
stack.append(20)
stack.pop()             # pop -> 20
stack[-1]               # peek -> 10
len(stack)              # size
```

6. Example Trace

push(10), push(20), push(30), pop(), push(40):

```
[] -> [10] -> [10,20] -> [10,20,30]
-> [10,20] (pop) -> [10,20,40]
```

7. Applications of Stack

- Function call management (Call Stack)
- Expression evaluation (Infix, Prefix, Postfix)
- Parentheses matching
- Backtracking (DFS, Maze, N-Queens)
- Undo operation in editors
- Browser history (back button)

8. Important Problems

- Valid Parentheses
- Next Greater Element
- Largest Rectangle in Histogram
- Evaluate Reverse Polish Notation
- Min Stack
- Basic Calculator

9. Complexities

Op	List (append/pop end)	Time
push	O(1)*	amortized
pop	O(1)	
peek	O(1)	
isEmpty	O(1)	
size	O(1)	

10. Code: Valid Parentheses

```
def is_valid(s):
    stack = []
    pairs = {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

7. DSA NOTES — 7. Queue

1. What is Queue?

A linear data structure that follows **FIFO** (First In First Out) order. Insertion at rear, deletion from front.

Think of a queue like a line in a bank — first person in is first served.

2. Representation

```
FRONT                       REAR
  v                           v
[10, 20, 30, 40]
Front = 0 (points to 10)
Rear  = 3 (points to 40)
```

3. Basic Operations

- enqueue(x) → insert x at rear
- dequeue() → remove & return front element
- front() → get front element
- isEmpty() / size()

4. Implementation (collections.deque)

```
from collections import deque
q = deque()
q.append(10)        # enqueue -> O(1)
q.append(20)
q.popleft()          # dequeue -> O(1) -> 10
q[0]                  # front element
len(q)                 # size
not q                   # isEmpty
```

5. Why not a plain list?

list.pop(0) is O(n) — shifts all elements. Always use `collections.deque` for O(1) enqueue/dequeue at both ends.

6. Example Trace

Queue size=5: enqueue(10,20,30), dequeue(), enqueue(40)

```
[] -> [10] -> [10,20] -> [10,20,30]
-> [20,30] (dequeue) -> [20,30,40]
```

7. Types of Queue

- Simple Queue — standard FIFO
- Circular Queue — rear wraps to front
- Priority Queue — dequeue by priority (heapq)
- Deque — insertion/deletion at both ends

8. Applications

- CPU Scheduling (Round Robin)
- Printer Spooling
- Breadth First Search (BFS)
- Handling requests in web servers
- Call Center Systems

9. Important Problems

- Implement Queue using List/Deque
- Circular Queue
- First Non-Repeating Character in Stream
- Sliding Window Maximum
- Rotten Oranges (BFS)
- Number of Islands (BFS)

10. Complexities

Op	deque	list
enqueue	O(1)	O(1)
dequeue	O(1)	O(n)
front	O(1)	O(1)
size	O(1)	O(1)

11. Priority Queue (heapq)

```
import heapq
pq = []
heapq.heappush(pq, 5)
heapq.heappush(pq, 1)
heapq.heappush(pq, 3)
heapq.heappop(pq)       # 1 (smallest first)
```

Queue may be simple, but it plays a big role in real life and DSA problems!

8. DSA NOTES — 8. Heap

1. What is Heap?

A special tree-based structure satisfying the Heap Property; a complete binary tree. Python implements heaps via **heapq** (min-heap only).

In Max Heap, the best element (largest) is always on top.

2. Heap Property

- Max Heap: parent >= children
- Min Heap: parent <= children (heapq default)

3. Types of Heap

- Min Heap: root = minimum element (heapq default)
- Max Heap: root = maximum (negate values in Python)

4. heapq Basics (Min Heap)

```
import heapq
h = []
heapq.heappush(h, 30)
heapq.heappush(h, 10)
heapq.heappush(h, 20)
heapq.heappop(h)     # 10 (smallest)
h[0]                  # peek min
heapq.heapify(list_)   # O(n) build heap
```

5. Max Heap in Python (negate trick)

```
import heapq
h = []
heapq.heappush(h, -30)
heapq.heappush(h, -10)
heapq.heappush(h, -20)
-heapq.heappop(h)     # 30 (largest)
```

6. Basic Operations

Op	Description	Time
heappush(h,x)	Insert x	O(log n)
heappop(h)	Remove & return min	O(log n)
h[0]	Peek min	O(1)
heapify(list)	Build heap in-place	O(n)
nlargest/nsmallest(k,h)	Top k	O(n log k)

7. Build Heap From Array

```
arr = [9, 4, 6, 1, 3, 5]
import heapq
heapq.heapify(arr)   # in-place, O(n)
# arr is now a valid min-heap
```

8. Heap Sort (using heapq)

```
def heap_sort(arr):
    heapq.heapify(arr)
    return [heapq.heappop(arr) for _ in range(len(arr))]
# Time: O(n log n)  Space: O(n)
```

9. K Largest / Smallest Elements

```
import heapq
heapq.nlargest(3, arr)     # top 3 largest
heapq.nsmallest(3, arr)    # top 3 smallest

# k largest using min-heap of size k
def k_largest(arr, k):
    h = arr[:k]
    heapq.heapify(h)
    for x in arr[k:]:
        if x > h[0]:
            heapq.heapreplace(h, x)
    return h
```

10. Priority Queue with Tuples

```
import heapq
pq = []
heapq.heappush(pq, (2, "task_b"))   # (priority, item)
heapq.heappush(pq, (1, "task_a"))
heapq.heappop(pq)   # (1, 'task_a') lowest priority first
```

11. Applications

- Priority Queue implementation
- Dijkstra's / Prim's Algorithm
- K largest/smallest elements
- Median in a stream (two heaps)
- Heap Sort
- Merge K sorted lists

Master Heap for scheduling & priority based problems!

9. DSA NOTES — 9. Set / Hash Set

1. What is a Set?

A collection of **unique** elements. In Python, `set` is implemented using a hash table.

Think of a Set as a box that keeps only unique elements — order doesn't matter.

2. Characteristics

- Stores only unique elements
- Unordered (no guaranteed order)
- Mutable; frozenset is immutable version
- Average O(1) for add/remove/contains
- Uses hashing internally

3. Common Set Operations

Operation	Description
add(x)	Insert x
remove(x)	Remove x (KeyError if absent)
discard(x)	Remove x safely
x in s	Check membership O(1) avg
len(s)	Size
clear()	Remove all elements

4. Set vs List

Feature	List	Set
Duplicates	Allowed	Not Allowed
Order	Maintains	Unordered
Access	By index O(1)	No index access
Search	O(n)	O(1) avg

5. Creating & Using Sets

```
s = set()
s = {1, 2, 3, 4, 5}
s.add(10)
s.add(10)               # duplicate -> ignored
s.remove(20)              # error if missing
s.discard(20)               # safe remove
print(3 in s)                 # membership O(1)
print(len(s))
```

6. Set Algebra (Union/Intersection)

```
a = {1, 2, 3}
b = {2, 3, 4}
a | b     # union        -> {1,2,3,4}
a & b     # intersection -> {2,3}
a - b     # difference   -> {1}
a ^ b     # symmetric diff-> {1,4}
a.issubset(b); a.issuperset(b)
```

7. Example: Remove Duplicates

```
arr = [4, 5, 4, 2, 5, 1, 2, 3]
unique = list(set(arr))
# {1, 2, 3, 4, 5}  (order not guaranteed)

# preserve order:
seen = set(); ordered = []
for x in arr:
    if x not in seen:
        seen.add(x); ordered.append(x)
```

8. Common Applications

- Removing duplicates from array/string
- Checking if an element exists
- Union & intersection of collections
- Solving problems involving unique data
- Counting distinct elements

9. Time Complexity (Average Case)

Op	set	Worst
Insert	O(1)	O(n)
Delete	O(1)	O(n)
Search	O(1)	O(n)
Traverse	O(n)	O(n)

10. set vs frozenset vs dict.fromkeys

Feature	set	frozenset
Mutable	Yes	No
Hashable	No	Yes (usable as dict key)
Order	Unordered	Unordered

11. Quick Tips

- Set stores only UNIQUE elements
- No indexing in Set
- Use set() to eliminate duplicates fast
- Great for membership testing (x in s)
- Very useful in competitive programming

Key Takeaway: Use Set when you need uniqueness and fast lookups!

10. DSA NOTES — 10. Map / Hash Map (dict)

1. What is a Dictionary (Map)?

A collection of key-value pairs. In Python, called `dict`. Keys are unique, values can repeat.

Think of a dict like a real dictionary — search a word (key), get its meaning (value) instantly!

2. Characteristics

- Stores key-value pairs
- Keys unique, values may repeat
- Insertion order preserved (Python 3.7+)
- Keys must be hashable (immutable types)
- Average O(1) time for basic operations

3. Common Dict Operations

Operation	Description
d[key] = value	Insert/update pair
d[key] / d.get(key)	Get value
key in d	Check if key exists
del d[key] / d.pop(key)	Remove pair
len(d)	Number of pairs
d.keys()/values()/items()	Views

4. Map vs Set vs List

Feature	dict	set	list
Stores	Key-Value	Unique elems	Ordered elems
Duplicates	Keys: No	No	Yes
Access	By key O(1)	No index	By index O(1)

5. Creating & Using Dicts

```
d = {}
d = {"Alice": 25, "Bob": 30}
d["Alice"] = 26            # update
d["Charlie"] = 100          # insert
age = d.get("Alice")          # 26
present = "Bob" in d            # True
del d["Bob"]                      # remove
print(len(d))                       # size
```

6. Iterating a Dict

```
for key in d:
    print(key, d[key])

for key, value in d.items():
    print(f"{key} -> {value}")

for value in d.values():
    print(value)
```

7. Example: Frequency Counting

```
from collections import Counter
s = "aabaccd"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
# {'a': 3, 'b': 1, 'c': 2, 'd': 1}

freq2 = Counter(s)   # same result, built-in
```

8. defaultdict / Counter

```
from collections import defaultdict, Counter
dd = defaultdict(int)
dd["x"] += 1               # no KeyError, defaults to 0

dd_list = defaultdict(list)
dd_list["a"].append(1)      # auto-creates empty list

c = Counter([1,1,2,3,3,3])
c.most_common(2)             # [(3,3), (1,2)]
```

9. Common Applications

- Frequency counting of elements
- Caching / memoization
- Graph adjacency list representation
- Grouping anagrams
- Fast lookups in competitive programming

10. Time Complexity (Average Case)

Op	dict	Worst
Insert	O(1)	O(n)
Delete	O(1)	O(n)
Search(get)	O(1)	O(n)
Iteration	O(n)	O(n)

11. Quick Tips

- Use dict.get(key, default) to avoid KeyError
- Use collections.Counter for frequency problems
- Use defaultdict to simplify grouping logic
- Dict = Key to Efficient Solutions!

11. DSA NOTES — 11. Linked List

1. What is a Linked List?

A linear data structure — a collection of nodes where each node contains data and a reference to the next node. Not stored contiguously.

Think of a Linked List as a chain where each link (node) points to the next link.

2. Node Structure

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

3. Types of Linked List

- Singly Linked List: head -> 10 -> 20 -> 30 -> None
- Doubly Linked List: has prev & next pointers
- Circular Singly Linked List: last node -> head
- Circular Doubly Linked List: both directions circular

4. Basic Operations

Op	Description
Traversal	Visit each node
Insertion	Add at beginning/end/pos
Deletion	Remove from beginning/end/pos
Search	Find node with given value
Length	Count total nodes

5. Insertion

```
# Insert at beginning - O(1)
def insert_at_head(head, data):
    new_node = Node(data)
    new_node.next = head
    return new_node

# Insert at end - O(n)
def insert_at_end(head, data):
    new_node = Node(data)
    if not head:
        return new_node
    temp = head
    while temp.next:
        temp = temp.next
    temp.next = new_node
    return head
```

6. Deletion

```
# Delete at beginning - O(1)
def delete_head(head):
    return head.next if head else None

# Delete at position - O(n)
def delete_at_pos(head, pos):
    if pos == 0:
        return head.next
    temp = head
    for _ in range(pos - 1):
        temp = temp.next
    temp.next = temp.next.next
    return head
```

7. Traversal

```
def traverse(head):
    temp = head
    while temp:
        print(temp.data, end=" ")
        temp = temp.next
# List: 10 -> 20 -> 30 -> None
# Output: 10 20 30
```

8. Search

```
def search(head, key):
    temp = head
    while temp:
        if temp.data == key:
            return True
        temp = temp.next
    return False
```

9. Length of Linked List

```
def length(head):
    cnt = 0
    temp = head
    while temp:
        cnt += 1
        temp = temp.next
    return cnt
```

10. Common Problems

- Reverse a Linked List
- Detect Loop (Floyd's Cycle Detection)
- Find Middle of Linked List
- Merge Two Sorted Linked Lists
- Remove Nth Node From End
- Check if Linked List is Palindrome

```
# Detect loop
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

11. Complexities

Op	Time	Space
Traversal	O(n)	O(1)
Search	O(n)	O(1)
Insert(pos)	O(n)	O(1)
Delete(pos)	O(n)	O(1)
Access(idx)	O(n)	O(1)

12. DSA NOTES — 12. Fast & Slow Pointers

1. What are Fast & Slow Pointers?

Also called Tortoise and Hare approach. Uses two pointers moving at different speeds.
Mainly used with Linked Lists in O(n) time, O(1) space.

Fast pointer is the hare, slow pointer is the tortoise!

2. Why is it Useful?

- Detect cycle in a Linked List
- Find the middle element
- Find the start of the cycle
- Solve problems in linear time, constant space

3. How it Works?

- Initialize slow & fast at head
- Move slow by 1 step
- Move fast by 2 steps
- Use meeting condition to find result

Time: O(n) | Space: O(1)

4. Template (Cycle Detection)

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True    # cycle found
    return False               # no cycle
```

5. Find Middle of Linked List

```
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow    # middle node
# For even length, returns 2nd middle
```

6. Find Start of Cycle

```
def detect_cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            break
    else:
        return None
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next
    return slow    # start of cycle
```

7. Length of Cycle

```
def cycle_length(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            length = 1
            temp = slow.next
            while temp != slow:
                temp = temp.next
                length += 1
            return length
    return 0
```

8. Remove Cycle From Linked List

```
def remove_cycle(head):
    start = detect_cycle_start(head)
    if not start:
        return
    temp = start
    while temp.next != start:
        temp = temp.next
    temp.next = None
```

9. When to Use?

- Any problem on Linked List involving cycle/middle
- To optimize space O(1) instead of extra data structures
- Frequently asked in interviews!

10. Quick Tips

- Always check fast and fast.next before moving
- Works only on Linked Lists
- Don't forget edge cases (empty, single, two nodes)
- Master this pattern — interview favorite!

Slow & steady wins the race, and finds the solution too!

1. What is Sorting?

The process of arranging elements in a particular order (ascending/descending).

Good sorting makes many problems easier & faster.

2. Key Points

- Sorting helps in efficient searching
- Different algorithms have different time/space complexity
- In-place sorting uses O(1) extra space
- Stable sorting keeps relative order of equal elements

3. Comparison Table

Algorithm	Best	Avg	Worst	Space	Stable
Bubble	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No

4. Built-in Sort (Timsort)

```
arr = [5, 1, 4, 2, 8]
arr.sort()                  # in-place, ascending
arr.sort(reverse=True)       # descending
sorted_arr = sorted(arr)       # returns new list
arr.sort(key=lambda x: -x)       # custom key
# Timsort: O(n log n) worst, stable
```

5. Bubble Sort (Stable)

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
# Best: O(n)  Worst: O(n^2)
```

6. Selection Sort (Not Stable)

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
# Time: O(n^2) all cases
```

7. Insertion Sort (Stable)

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
# Best: O(n)  Worst: O(n^2)
```

8. Merge Sort (Stable)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(l, r):
    res, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if l[i] <= r[j]:
            res.append(l[i]); i += 1
        else:
            res.append(r[j]); j += 1
    return res + l[i:] + r[j:]
# Time: O(n log n)  Space: O(n)
```

9. Quick Sort (Not Stable)

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left  = [x for x in arr if x < pivot]
    mid   = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)
# Avg: O(n log n)  Worst: O(n^2)
```

10. Counting Sort (Stable)

```
def counting_sort(arr):
    if not arr: return arr
    mx = max(arr)
    count = [0]*(mx+1)
    for x in arr: count[x] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i]*c)
    return out
# Time: O(n+k)  Space: O(n+k)
```

11. When to Use What?

- Nearly sorted / small array -> Insertion Sort
- Need stability + O(n log n) -> Merge Sort
- In-place + fast avg case -> Quick Sort (arr.sort())
- Guaranteed O(n log n) + in-place -> Heap Sort

14. DSA NOTES — 14. Searching

1. What is Searching?

The process of finding the position (index) of a target element in a collection.

Searching is simple, but the right choice makes your code fast!

2. Linear Search (Sequential)

Check each element one by one until target found or list ends.

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
# Time: O(n)  Space: O(1)
```

3. Binary Search (Sorted Array)

Repeatedly divide search space in half.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
# Time: O(log n)  Space: O(1)
```

4. Using Built-in bisect

```
import bisect
arr = [2, 5, 8, 12, 16, 23, 38, 56]
bisect.bisect_left(arr, 16)    # index of 16 (or insert pos)
bisect.bisect_right(arr, 16)   # insert pos after equals
bisect.insort(arr, 20)         # insert keeping sorted
```

5. Comparison Table

Feature	Linear	Binary
Requires Sorted	No	Yes
Approach	Check one by one	Divide & conquer
Time (Best)	O(1)	O(1)
Time (Avg/Worst)	O(n)	O(log n)
Use Case	Small/unsorted	Large/sorted

6. When to Use What?

- Use Linear when: data unsorted, size small, simplicity preferred
- Use Binary when: data sorted, size large, faster search required

Always check if the array is sorted before using Binary Search!

7. Binary Search Variations

- First Occurrence (Lower Bound) — bisect_left
- Last Occurrence (Upper Bound) — bisect_right - 1
- Search Insert Position — bisect_left
- Count Occurrences — bisect_right - bisect_left

```
arr=[1,2,2,2,3,4,5]; target=2
first = bisect.bisect_left(arr, target)    # 1
last  = bisect.bisect_right(arr, target)-1 # 3
count = bisect.bisect_right(arr,target) - bisect.bisect_left(arr,target) # 3
```

8. Edge Cases

- Empty array
- Single element array
- Target not present
- All elements same
- Target smaller/greater than all elements

9. Time Complexity Proof (Binary Search)

Each step halves the search space. After k steps: n/2^k = 1 → k = log2(n)

Time Complexity = O(log n)

10. Practice Problems

- Search in a sorted array
- Search Insert Position
- First & Last Position of Element
- Count Occurrences in Sorted Array
- Row with minimum 1's (Binary Search on answer)
- Square root of a number (Binary Search)

15. DSA NOTES — 15. Binary Search on Answer

1. What is Binary Search on Answer?

A technique where instead of binary searching the array, we binary search on the **possible answers** (answer space).

We search the answer space, not the array!

2. When to Use?

- When the answer lies in a range
- When the answer is monotonic (if X works, any Y>=X works)
- When brute force is hard but a solution can be verified in O(n)

3. How Does It Work?

- 1. Define answer space [low, high]
- 2. While low <= high: mid = (low+high)//2
- 3. Check feasibility using is_possible(mid)
- 4. If possible, try better answer; else try bigger/smaller
- 5. Return the best answer

4. Template

```
def binary_search_on_answer(low, high, is_possible):
    ans = -1
    while low <= high:
        mid = (low + high) // 2
        if is_possible(mid):
            ans = mid
            high = mid - 1     # try smaller/better
        else:
            low = mid + 1        # try bigger
    return ans
```

5. Example: Aggressive Cows

Place C cows in N stalls to maximize the minimum distance between any two cows.

```
def is_possible(stalls, dist, cows):
    count, last = 1, stalls[0]
    for pos in stalls[1:]:
        if pos - last >= dist:
            count += 1
            last = pos
    return count >= cows

def aggressive_cows(stalls, cows):
    stalls.sort()
    low, high, ans = 0, stalls[-1]-stalls[0], 0
    while low <= high:
        mid = (low + high) // 2
        if is_possible(stalls, mid, cows):
            ans = mid; low = mid + 1
        else:
            high = mid - 1
    return ans
```

6. Example: Koko Eating Bananas

Find minimum eating speed K so Koko eats all bananas within H hours.

```
import math
def is_possible(piles, k, h):
    hours = sum(math.ceil(p / k) for p in piles)
    return hours <= h

def min_eating_speed(piles, h):
    low, high, ans = 1, max(piles), max(piles)
    while low <= high:
        mid = (low + high) // 2
        if is_possible(piles, mid, h):
            ans = mid; high = mid - 1
        else:
            low = mid + 1
    return ans
```

7. Example: Min Days for M Bouquets

```
def is_possible(bloom, days, m, k):
    count = flowers = 0
    for d in bloom:
        if d <= days:
            flowers += 1
            if flowers == k:
                count += 1; flowers = 0
        else:
            flowers = 0
    return count >= m

def min_days(bloom, m, k):
    low, high, ans = min(bloom), max(bloom), -1
    while low <= high:
        mid = (low + high) // 2
        if is_possible(bloom, mid, m, k):
            ans = mid; high = mid - 1
        else:
            low = mid + 1
    return ans
```

8. Choosing Boundaries

Problem Type	low	high
Maximum value	0 or 1	large value
Minimum value	0 or 1	large value
Answer in array range	min(arr)	max(arr)
Speed/Rate problems	1	max possible speed

9. Common Mistakes

- Wrong boundaries
- Wrong condition in is_possible()
- Forgetting edge cases
- Infinite loop (wrong mid update)
- Returning wrong answer (best vs last)

10. Quick Tips

- Always define what your answer means
- Write is_possible(x) cleanly, test separately
- Check monotonicity of your condition

16. DSA NOTES — 16. Bit Manipulation

1. What is Bit Manipulation?

Performing operations directly on bits. Helps optimize solutions; used in low-level programming, cryptography, competitive programming.

> 💡 *Think in bits, solve it like a pro!*

2. Number System Recap

Decimal	Binary
0	0000
1	0001
2	0010
5	0101
15	1111

3. Basic Bitwise Operators

```
a, b = 5, 3        # 0101, 0011
a & b    # AND -> 1  (0001)
a | b    # OR  -> 7  (0111)
a ^ b    # XOR -> 6  (0110)
~a       # NOT -> -6 (2's complement)
a << 1   # Left shift  -> 10 (1010)
a >> 1   # Right shift -> 2  (0010)
```

4. Properties

- a & a = a, a | a = a
- a & 0 = 0, a | 0 = a
- a & 1s = a, a | 1s = 1s
- a ^ a = 0, a ^ 0 = a
- a ^ a ^ b = b
- ~(~a) = a

5. Set, Clear, Toggle a Bit

```
n = 10           # 1010
i = 1
is_set = (n & (1 << i)) != 0     # check bit i
n |= (1 << i)                      # set bit i
n &= ~(1 << i)                     # clear bit i
n ^= (1 << i)                      # toggle bit i
```

6. Power of Two

```
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
# n=8 (1000) -> True
# n=6 (0110) -> False
```

7. Count Set Bits (1's)

```
def count_set_bits(n):
    count = 0
    while n:
        n &= (n - 1)     # remove last set bit
        count += 1
    return count

bin(13).count('1')     # built-in way -> 3
```

8. Clear / Isolate Last Set Bit

```
n = 20                  # 10100
clear_last = n & (n - 1)   # 10000 (16)
isolate_last = n & (-n)      # 00100 (4)
```

9. Swap Two Numbers (XOR)

```
a, b = 5, 9
a = a ^ b
b = a ^ b     # now b = original a
a = a ^ b     # now a = original b
# Pythonic way: a, b = b, a
```

10. Check Odd/Even & Mul/Div by 2

```
n & 1 == 1     # odd
n & 1 == 0     # even
n << 1          # multiply by 2
n >> 1          # divide by 2 (integer)
```

11. Negative Numbers (2's Complement)

```
# Python ints are arbitrary precision;
# use bin(n & 0xFF) to view fixed-width form
n = -5
bin(n & 0xFF)   # '0b11111011' (8-bit view)
```

12. Applications

- Count set bits
- Check power of two
- Generate subsets using bitmasks
- XOR based problems (Single Number)
- Find missing number in array

13. Common Bit Masks

```
ALL_SET   = (1 << 32) - 1     # 32 bits all 1
ONLY_ITH  = 1 << i               # only ith bit set
ALL_EXCEPT_I = ~(1 << i)
```

Bit Manipulation may look tricky first, but with practice it becomes super easy!

17. DSA NOTES — 17. Binary Tree

1. What is Binary Tree?

A hierarchical data structure where each node has at most two children: left and right. Topmost node is Root.

A Binary Tree is either empty or a root with two disjoint binary trees (left & right subtree).

2. Node Structure

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

3. Terminologies

- Root: topmost node
- Leaf: node with no children
- Height of Node: edges on longest path to leaf
- Height of Tree: height of root node
- Level of root = 0

4. Types of Binary Tree

- Full: every node has 0 or 2 children
- Complete: all levels filled except possibly last (left to right)
- Perfect: all internal nodes have 2 children, all leaves same level
- Skewed: every node has only one child

5. Traversals (Recursive)

```
def preorder(root):     # Root, Left, Right
    if not root: return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def inorder(root):      # Left, Root, Right
    if not root: return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def postorder(root):    # Left, Right, Root
    if not root: return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")
```

6. Level Order Traversal (BFS)

```
from collections import deque
def level_order(root):
    if not root: return
    q = deque([root])
    while q:
        node = q.popleft()
        print(node.data, end=" ")
        if node.left:  q.append(node.left)
        if node.right: q.append(node.right)
```

7. Height of Binary Tree

```
def height(root):
    if not root:
        return -1     # empty tree height = -1
    return 1 + max(height(root.left), height(root.right))
```

8. Count Nodes

```
def count_nodes(root):
    if not root:
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)
# Time: O(n)  Space: O(h)
```

9. Diameter of Binary Tree

```
def diameter(root):
    best = [0]
    def depth(node):
        if not node: return 0
        l, r = depth(node.left), depth(node.right)
        best[0] = max(best[0], 1 + r)
        return 1 + max(l, r)
    depth(root)
    return best[0]
```

10. Check Identical / Mirror Trees

```
def is_identical(a, b):
    if not a and not b: return True
    if not a or not b: return False
    return (a.data == b.data and
            is_identical(a.left, b.left) and
            is_identical(a.right, b.right))

def is_mirror(a, b):
    if not a and not b: return True
    if not a or not b: return False
    return (a.data == b.data and
            is_mirror(a.left, b.right) and
            is_mirror(a.right, b.left))
```

11. Common Applications

- Expression Tree
- Binary Search Tree
- Huffman Coding
- Heaps / Priority Queues
- File System / XML Parsing

18. DSA NOTES — 18. Binary Search Tree

1. What is BST?

A Binary Search Tree is a binary tree where for every node N: all keys in left subtree < N.data < all keys in right subtree.

BST is the base of many advanced data structures!

2. Properties

- Left Subtree < Root < Right Subtree
- Inorder traversal gives sorted sequence
- No duplicate keys (by default)
- Search/Insert/Delete take O(h) time
- Balanced BST: h=O(log n); Skewed: h=O(n)

3. Search in BST

```
def search(root, key):
    while root:
        if key == root.data:
            return root
        root = root.left if key < root.data else root.right
    return None
# Time: O(h)  Space: O(1) iterative
```

4. Insert in BST

```
def insert(root, key):
    if not root:
        return Node(key)
    if key < root.data:
        root.left = insert(root.left, key)
    elif key > root.data:
        root.right = insert(root.right, key)
    return root   # duplicates ignored
```

5. Delete in BST

```
def min_value_node(node):
    while node.left:
        node = node.left
    return node

def delete_node(root, key):
    if not root: return root
    if key < root.data:
        root.left = delete_node(root.left, key)
    elif key > root.data:
        root.right = delete_node(root.right, key)
    else:
        if not root.left:  return root.right
        if not root.right: return root.left
        succ = min_value_node(root.right)
        root.data = succ.data
        root.right = delete_node(root.right, succ.data)
    return root
```

6. Find Min & Max in BST

```
def find_min(root):
    while root.left: root = root.left
    return root

def find_max(root):
    while root.right: root = root.right
    return root
# Time O(h)  Space O(1)
```

7. Validate BST

```
def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root:
        return True
    if not (low < root.data < high):
        return False
    return (is_valid_bst(root.left, low, root.data) and
            is_valid_bst(root.right, root.data, high))
```

8. Height of BST

```
def height(root):
    if not root: return -1
    return 1 + max(height(root.left), height(root.right))
# Time: O(n)  Space: O(h)
```

9. Construct BST From Sorted Array

```
def sorted_array_to_bst(arr, l, r):
    if l > r: return None
    mid = (l + r) // 2
    root = Node(arr[mid])
    root.left = sorted_array_to_bst(arr, l, mid-1)
    root.right = sorted_array_to_bst(arr, mid+1, r)
    return root
# Time: O(n)  Space: O(log n)
```

10. Inorder Successor / Predecessor

```
def successor(root, key):
    succ = None
    while root:
        if key < root.data:
            succ = root; root = root.left
        else:
            root = root.right
    return succ
# Predecessor: mirror logic (go right when key > root.data)
```

11. Applications

- Searching efficiently
- Maintaining sorted data
- Implementing Sets and Maps
- Range queries
- Basis for AVL, Red-Black, Splay Trees

Master BST, and half of the DSA trees are yours!

1. What is Graph?

A non-linear data structure consisting of vertices (nodes) and edges connecting pairs of vertices.

Everything is connected! That's why graphs are everywhere.

2. Terminologies

- Vertex/Node: a point in the graph
- Edge: connection between two vertices
- Degree: number of edges incident on a vertex
- Path: sequence of connected vertices
- Cycle: path that starts and ends at same vertex

3. Types of Graphs

- Undirected Graph
- Directed Graph (Digraph)
- Weighted Graph
- Unweighted Graph

4. Graph Representation (Adjacency List)

```
graph = {
    1: [2, 4],
    2: [1, 3],
    3: [2, 4],
    4: [1, 3],
}
# Or using defaultdict(list) for building dynamically
from collections import defaultdict
g = defaultdict(list)
g[1].append(2)
g[2].append(1)   # undirected: add both directions
```

5. Adjacency Matrix

```
n = 4
matrix = [[0]*n for _ in range(n)]
matrix[0][1] = matrix[1][0] = 1   # edge 1-2
# Space: O(V^2) matrix, O(V+E) list
```

6. Handshaking Lemma

In an undirected graph: sum of degrees of all vertices = 2 × (number of edges). Every edge contributes 1 to each endpoint's degree.

7. Directed Graph Concepts

- Indegree: edges coming into a vertex
- Outdegree: edges going out from a vertex
- Total degree = Indegree + Outdegree

8. BFS (Breadth First Search)

```
from collections import deque
def bfs(graph, start):
    visited = {start}
    q = deque([start])
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr)
                q.append(nbr)
    return order
# Time: O(V+E)  Uses Queue
```

9. DFS (Depth First Search)

```
def dfs(graph, start, visited=None, order=None):
    if visited is None: visited, order = set(), []
    visited.add(start)
    order.append(start)
    for nbr in graph[start]:
        if nbr not in visited:
            dfs(graph, nbr, visited, order)
    return order
# Time: O(V+E)  Uses Stack/Recursion
```

10. BFS vs DFS

BFS	DFS
Uses Queue	Uses Stack/Recursion
Level order traversal	Depth wise traversal
Shortest path (unweighted)	May not find shortest path
Good for shortest path	Good for topo sort, cycle detection

11. Cycle Detection

```
# Undirected (via DFS + parent check)
def has_cycle_undirected(graph, node, visited, parent):
    visited.add(node)
    for nbr in graph[node]:
        if nbr not in visited:
            if has_cycle_undirected(graph, nbr, visited, node):
                return True
        elif nbr != parent:
            return True
    return False
```

12. Topological Sort (DAG)

```
def topo_sort(graph, n):
    visited, stack = set(), []
    def dfs(u):
        visited.add(u)
        for v in graph[u]:
            if v not in visited:
                dfs(v)
        stack.append(u)
    for node in range(n):
        if node not in visited:
            dfs(node)
    return stack[::-1]
```

13. Applications

- Social networks
- Google Maps / GPS Navigation
- Dependencies / Scheduling
- Network routing
- Recommendation systems

1. What is Dynamic Programming?

DP solves problems by breaking them into overlapping subproblems, solving each once, and storing the result to avoid recomputation.

Think Recursively. Solve Smartly. Save Repetition!

2. Key Principles

- Overlapping Subproblems: subproblems solved repeatedly
- Optimal Substructure: optimal solution built from optimal sub-solutions
- Memoization: Top-Down using recursion + cache
- Tabulation: Bottom-Up using iteration + table

3. Approaches

- Memoization (Top-Down): recursion, store in dict/array
- Tabulation (Bottom-Up): iterative, build table from base cases

4. General Template (Memoization)

```
from functools import lru_cache

@lru_cache(maxsize=None)
def solve(i, j):
    if base_case:
        return base_answer
    ans = ...            # try all choices
    return ans
```

5. Common DP Patterns

Pattern	Examples
1D DP	Fibonacci, Climbing Stairs
2D DP	Grid Paths, LCS, Edit Distance
Knapsack DP	0/1 Knapsack, Subset Sum
DP on Sequences	LIS, LCS, Palindrome Partition
Bitmask DP	TSP, Assignments

6. Example: Fibonacci Number

```
# Memoization
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

# Tabulation
def fib_tab(n):
    dp = [0]*(n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
# Time: O(n)  Space: O(n) / O(1) optimized
```

7. Example: Climbing Stairs

```
def climb_stairs(n):
    if n <= 1: return 1
    a, b = 1, 1
    for _ in range(2, n+1):
        a, b = b, a + b
    return b
# Time: O(n)  Space: O(1)
```

8. Example: 0/1 Knapsack

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
# Time: O(n*W)  Space: O(n*W)
```

9. Example: Longest Common Subsequence

```
def lcs(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for j in range(1, m+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[n][m]
# Time: O(n*m)  Space: O(n*m)
```

10. Space Optimization

- Convert 2D DP to 1D DP where possible
- Use rolling arrays (keep only last row/col)
- Use variables if only previous state needed

```
# Fibonacci with O(1) space
def fib_opt(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

11. Common DP Problems

- Fibonacci, Climbing Stairs, House Robber
- Unique Paths (Grid), Min Path Sum
- Coin Change (Min Coins/Ways)
- 0/1 Knapsack, Subset Sum
- LIS, LCS, Edit Distance
- Matrix Chain Multiplication
- Palindrome Partitioning

12. When to Use DP?

- Problem has overlapping subproblems
- Recursive solution recomputes same states
- Optimal solution built from sub-solutions
- Brute force is exponential/too slow

If recursion is TLE -> Think DP! DP turns exponential into polynomial.